

# DNN EXAM SURVIVAL PACK

Detailed Theory + Formulas + Step-by-Step Numericals + Likely Questions  
+ Quiz

Built from your uploaded DNN course material  
Modules 1-5 + the uploaded DFNN numerical practice material.

## WHAT THIS PACK IS FOR

A revision companion you can keep with you before the exam. The emphasis is on understanding why each formula exists, how the concepts connect, and how to solve the numerical questions step by step.

Priority key: ★★★ = high yield    ★★★★★ = must master

The explanations follow the terminology and progression used in the uploaded material. Where a concept is expanded for understanding, it is presented as an explanation of the course idea rather than as a replacement for it.

## HOW TO USE THIS PACK

The course is easier to remember if you see it as one continuous story rather than as separate chapters. Module 1 establishes the learning framework. Module 2 introduces the neuron and perceptron. Module 3 turns a single neuron into a regression model. Module 4 turns it into classification models. Module 5 stacks these ideas into a deep feed-forward network and adds backpropagation.

The central loop is:

**Data → Model → Prediction → Loss → Gradient → Parameter update → Better prediction**

When studying, always ask three questions: (1) What goes into the model? (2) What comes out? (3) How does the model know how to improve? Those three questions connect almost every formula in the course.

### Suggested order for revision:

- 1 Concept foundation: AI/ML/DL, supervised learning, data/model/loss/learning algorithm.
- 2 Neuron foundation: artificial neuron, weights, bias, perceptron, logic gates and linear separability.
- 3 Key limitation: XOR and why non-linearity is necessary.
- 4 Regression: linear model, MSE, gradient descent and evaluation.
- 5 Classification: sigmoid, BCE, softmax, categorical cross-entropy and evaluation.
- 6 DFNN: layer-wise forward propagation, parameter counting and mini-batches.
- 7 Backpropagation: error signal, chain rule, parameter gradients and updates.
- 8 Final pass: formulas, numerical templates, likely questions and quiz.

### SOURCE BASIS

- DNN Module 1 — Introduction to Deep Learning
- DNN Module 2 — Artificial Neural Network / Perceptron
- DNN Module 3 — Linear Neural Networks for Regression
- DNN Module 4 — Linear Neural Networks for Classification
- DNN Module 5 — Deep Feed-Forward Neural Networks
- Uploaded DFNN Numerical practice material

# 1. MODULE 1 — INTRODUCTION TO DEEP LEARNING

★ Deep learning is introduced in your material as a type of machine learning based on artificial neural networks in which multiple layers of processing extract progressively higher-level features from data. The word deep therefore refers to successive layers of increasingly meaningful representations, not merely to having a large number of neurons.

A useful way to understand this is to imagine a hierarchy. At an early stage, the model may represent relatively simple patterns. Later layers combine earlier representations into more meaningful structures. The course material explicitly connects the number of layers contributing to the model with its depth.

This is why saying only 'deep learning has many layers' is incomplete. The important idea is that the layers are successive transformations that build representations.

## Deep Learning $\subset$ Machine Learning $\subset$ Artificial Intelligence

AI is the broadest idea in the module: making machines perform tasks associated with intelligence. Machine learning is an approach to AI in which a machine learns from experience/data. Deep learning is a specific ML approach using neural networks and layered representations.

EXAM TIP: If asked 'Why is it called deep learning?', mention successive layers of increasingly meaningful representations and depth = number of layers contributing to the model.

## WHY DEEP LEARNING?

Your material highlights the combination of large amounts of data, especially unstructured data such as images, text, audio and video; cheaper computation and storage; better sensors; scalability; and automated feature generation. The important conceptual shift is that a neural network can learn useful features from examples rather than requiring every useful feature to be manually designed.

For example, in an image problem, a system can learn intermediate representations that are useful for recognizing objects. The course's applications table connects tasks such as image recognition, object detection, speech recognition, translation, recommendation and autonomous driving with neural-network models.

## FOUR CORE COMPONENTS OF A LEARNING PROBLEM ★★★★★

| Component          | Question it answers                    | What it means in practice                        |
|--------------------|----------------------------------------|--------------------------------------------------|
| Data               | What do we learn from?                 | Examples, features and targets/labels.           |
| Model              | How do we transform input into output? | A parameterized mathematical mapping.            |
| Objective / Loss   | How wrong is the model?                | A numerical measure where lower is better.       |
| Learning algorithm | How do parameters improve?             | An optimization method such as gradient descent. |

A very important connection: parameters belong to the model, while the learning algorithm changes those parameters using information from the loss. Training is therefore not the same as inference. During training, we need targets and calculate loss/gradients. During inference, the trained parameters are simply used to make predictions.

## SUPERVISED, UNSUPERVISED AND REINFORCEMENT LEARNING ★★★★★

| Type          | Core idea                                                               | Examples in the uploaded material              |
|---------------|-------------------------------------------------------------------------|------------------------------------------------|
| Supervised    | Learn from input + label pairs.                                         | Regression, classification                     |
| Unsupervised  | Use data/features without supplied labels.                              | Clustering, density estimation, synthetic data |
| Reinforcement | Agent interacts with an environment and learns through trial and error. | Reinforcement learning settings                |

For this DNN course, supervised learning is particularly important because Modules 3 and 4 formulate regression and classification as supervised learning tasks.

## 2. MODULE 2 — ARTIFICIAL NEURAL NETWORKS & PERCEPTRON

★ The course introduces the artificial neuron as a simplified mathematical processing element inspired by the biological neuron. The analogy is not that the artificial neuron reproduces the brain biologically. Rather, it captures an important computational idea: multiple inputs are combined, their influences are weighted, and a response is produced.

| Biological idea | Artificial-network analogy | Interpretation                                 |
|-----------------|----------------------------|------------------------------------------------|
| Dendrites       | Inputs                     | Signals/features enter the neuron.             |
| Synapses        | Weights                    | Connection strengths encode learned influence. |
| Cell body       | Computation                | Inputs are combined and transformed.           |
| Axon            | Output                     | The neuron passes its result onward.           |

### ARTIFICIAL NEURON ★★★★★

Suppose the neuron receives features  $x_1, x_2, \dots, x_n$ . Each feature has a weight  $w_i$ . The neuron first calculates a weighted sum. The bias shifts this sum, and then an activation function transforms it.

$$z = \sum_i w_i x_i + b$$

$$\hat{y} = f(z) = f(\sum_i w_i x_i + b)$$

The key idea behind the weight is influence. A larger positive weight tends to make the corresponding input push  $z$  upward; a negative weight tends to push it downward. The bias provides an additional adjustable offset. The activation then determines how the neuron converts that combined signal into its output.

### CONNECTIONISM ★★★

The module describes neural networks as connectionist machines: knowledge is represented in the connections between processing elements, and learning changes the connection strengths. This is a useful conceptual bridge to later modules because the parameters we call weights are exactly the quantities training attempts to adjust.

### PERCEPTRON ★★★★★

The perceptron is presented as the simplest form of artificial neuron and a single-layer linear classifier. It computes a weighted sum and applies a threshold/step decision.

$$\hat{y} = 1 \text{ if } \sum_i w_i x_i + b \geq 0; \text{ otherwise } 0$$

The uploaded material also notes the classical  $\{-1, +1\}$  convention. Modern logic-gate examples in the same material use  $\{0, 1\}$ . The two conventions are related, but the chosen output convention changes the threshold/bias arithmetic.

### WHY LOGIC GATES MATTER

AND and OR are useful because their input-output relationships can be separated by a straight line in the two-dimensional input space. The module gives example perceptron parameters for these gates. For AND, one valid solution shown is  $w_0 = -1$ ,  $w_1 = 0.75$ ,  $w_2 = 0.75$ . For OR, one valid solution is  $w_0 = -1$ ,  $w_1 = 2$ ,  $w_2 = 2$ .

The important lesson is not the specific numbers. It is that the weights and bias define a decision boundary, and a suitable choice of those parameters can separate linearly separable truth-table classes.

## PERCEPTRON LEARNING ALGORITHM ★★★★★

$$\mathbf{w}_i(\text{new}) = \mathbf{w}_i(\text{old}) + \eta(\mathbf{t} - \hat{\mathbf{y}})\mathbf{x}_i$$

$$\mathbf{b}(\text{new}) = \mathbf{b}(\text{old}) + \eta(\mathbf{t} - \hat{\mathbf{y}})$$

Interpret the update in words: new parameter = old parameter + learning rate × error × input. If the prediction is already correct, the course algorithm leaves the parameters unchanged. If the prediction is wrong, the error term determines the direction and size of the correction.

The module states the perceptron convergence result: if the training data are linearly separable, the perceptron learning algorithm converges in a finite number of steps.

EXAM TIP: For a perceptron numerical, make a table with columns: example → weighted sum → prediction → target → error → updated weights/bias. Do not skip the intermediate weighted sum.

### 3. XOR, LINEAR SEPARABILITY & WHY NON-LINEARITY IS ESSENTIAL

#### XOR ★★★★★

| $x_1$ | $x_2$ | XOR |
|-------|-------|-----|
| 0     | 0     | 0   |
| 0     | 1     | 1   |
| 1     | 0     | 1   |
| 1     | 1     | 0   |

The two positive examples are (0,1) and (1,0), while the two negative examples are (0,0) and (1,1). No single straight line can put the positive pair on one side and the negative pair on the other. That is the geometric reason a single perceptron fails.

**Decision boundary of a single neuron:  $w_1x_1 + w_2x_2 + b = 0$**

In 2D this equation is a line. In higher dimensions it becomes a hyperplane. Therefore the limitation of a single perceptron is fundamentally a limitation on the shape of the decision boundary it can construct.

#### LINEAR SEPARABILITY ★★★★★

The uploaded module defines two sets of points as linearly separable in  $n$ -dimensional space if an  $(n-1)$ -dimensional hyperplane can separate them. In 2D, this means a straight line can separate the classes.

XOR is therefore the canonical example of a non-linearly separable problem. The solution is to create additional intermediate representations with a multilayer network.

#### MULTILAYER PERCEPTRON — THE INTUITION ★★★★★

A multilayer network does not merely add more arithmetic. Its hidden units transform the original input space into a new representation. The uploaded DFNN material explains the XOR solution using a 2-input, 2-hidden-unit, 1-output architecture: the hidden layer creates new features, and the output layer combines them.

This gives a powerful way to think about deep learning: hidden layers can change the representation so that a problem that was difficult to separate in the original space becomes easier to separate in a learned space.

#### WHY NON-LINEAR ACTIVATIONS? ★★★★★

Suppose every layer were purely linear. Consider two layers:

$$h = W_1x + b_1$$

$$y = W_2h + b_2$$

Substituting  $h$  into the second equation gives:

$$y = W_2(W_1x + b_1) + b_2 = (W_2W_1)x + (W_2b_1 + b_2)$$

The result is still just another linear/affine transformation of  $x$ . So simply stacking linear layers does not give the network the ability to represent arbitrary nonlinear relationships.

A nonlinear activation breaks this collapse. After every layer, the model can bend the representation instead of only rotating/scaling/shifting it linearly.

**Linear transformation → Nonlinear activation → Linear transformation →  
Nonlinear activation → ...**

EXAM TIP: A strong theory answer is: Without nonlinear activations, the composition of linear layers is still linear, so depth would not provide the desired nonlinear expressive power.

## 4. ACTIVATION FUNCTIONS — CONCEPT + FORMULAS

Activation functions are the mechanism that introduces non-linearity into the network. They also determine the numerical range and behavior of the neuron's output.

### SIGMOID ★★★★★

$$\sigma(z) = 1 / (1 + e^{(-z)})$$

Sigmoid maps every real-valued  $z$  into the interval  $(0,1)$ . That makes it useful when the output needs to be interpreted as a probability-like quantity for a binary classification problem.

Its shape is S-like. Very negative inputs are pushed close to 0; very positive inputs are pushed close to 1. Around  $z = 0$ , the output is 0.5. This is why a 0.5 classification threshold corresponds to a zero-valued logit.

$$\sigma'(z) = \sigma(z)[1 - \sigma(z)]$$

The derivative is especially convenient because once you know the sigmoid output, you can calculate its derivative without evaluating another exponential.

Example: if  $\sigma(z)=0.8$ , then  $\sigma'(z)=0.8(1-0.8)=0.16$ .

### TANH ★★★

$$\tanh(z) = (e^z - e^{(-z)}) / (e^z + e^{(-z)})$$

$$\tanh'(z) = 1 - \tanh^2(z)$$

Tanh maps values to  $(-1,1)$ , so unlike sigmoid it is zero-centered. The course comparison notes moderate vanishing-gradient behavior and higher computational cost than ReLU.

### RELU ★★★★★

$$\text{ReLU}(z) = \max(0, z)$$

$$\text{ReLU}'(z) = 1 \text{ if } z > 0; 0 \text{ if } z \leq 0$$

ReLU is conceptually simple: it keeps positive values and clips negative values to zero. That makes it computationally cheap and gives a gradient of 1 on the positive side, so positive signals can pass through without the repeated shrinking associated with sigmoid/tanh.

The uploaded material also highlights sparse activation: many neurons can output exactly zero. The main caveat is the dying ReLU problem, where a neuron can remain in the negative region and consequently produce zero output/gradient. The material mentions Leaky ReLU, PReLU and ELU as variants.

| Property               | Sigmoid       | Tanh         | ReLU                | Leaky ReLU           |
|------------------------|---------------|--------------|---------------------|----------------------|
| Range                  | $(0,1)$       | $(-1,1)$     | $[0,\infty)$        | $(-\infty,\infty)$   |
| Zero-centered          | No            | Yes          | No                  | Yes                  |
| Vanishing gradient     | High          | Moderate     | No on positive side | No on positive side  |
| Typical role in course | Binary output | Hidden layer | Hidden layer        | Hidden layer variant |

## HOW TO CHOOSE THE ACTIVATION

For this course's core examples, think of the output activation as task-dependent: identity for regression, sigmoid for binary classification, and softmax for multiclass classification. For hidden layers in DFNN, ReLU is the default emphasized in the material.

COMMON TRAP: Do not write 'ReLU is used everywhere.' The course distinguishes hidden-layer activations from output-layer activations.

## 5. MODULE 3 — LINEAR NEURAL NETWORKS FOR REGRESSION

Regression asks a continuous-output question: how much, how many, or what numerical value? The module examples include house price, temperature and stock-price prediction.

### LINEAR REGRESSION ★★★★★

$$\hat{y} = w_0 + w_1x_1 + w_2x_2 + \dots + w_dx_d$$

**Equivalent vector form:**  $\hat{y} = \mathbf{w}^T \mathbf{x} + b$

A linear regression model assumes that the target can be approximated by a linear combination of the input features. In neural-network language, one neuron with identity activation implements this model.

**Identity activation:**  $f(z) = z$

Why identity? Because regression targets are continuous and may take any real value. An activation such as sigmoid would artificially restrict the output to (0,1), which is inappropriate for an unrestricted quantity such as price or temperature.

### DESIGN MATRIX & BIAS AUGMENTATION ★★★★★

The module introduces an augmented feature vector by placing a 1 in front of the features. This allows the bias to be treated as another weight:

$$\tilde{\mathbf{x}} = [1, x_1, \dots, x_d]^T$$

$$\mathbf{w} = [w_0, w_1, \dots, w_d]^T$$

$$\hat{y} = \mathbf{X}\mathbf{w}$$

This notation is not just cosmetic. It lets all examples be processed as matrix operations and makes the dimensions explicit.

### SQUARED ERROR / MSE ★★★★★

**Single-example loss:**  $\ell = \frac{1}{2}(\hat{y} - y)^2$

**Course objective:**  $J(\mathbf{w}) = (1/(2N)) \|\mathbf{X}\mathbf{w} - \mathbf{y}\|^2$

The square has two important effects. First, positive and negative errors do not cancel because the square is non-negative. Second, large errors are penalized more strongly than small errors because the penalty grows quadratically.

The module also notes that for linear regression this objective is convex, meaning the error surface has a single global minimum. That makes the optimization picture especially clean.

### GRADIENT DESCENT ★★★★★

$$\mathbf{w}(\text{new}) = \mathbf{w}(\text{old}) - \eta \nabla J(\mathbf{w})$$

The gradient points in the direction of greatest local increase of the loss. Therefore subtracting the gradient moves the parameters toward decreasing loss. The learning rate  $\eta$  controls how large a step we take.

$$\nabla J(\mathbf{w}) = (1/N) \mathbf{X}^T(\mathbf{X}\mathbf{w} - \mathbf{y})$$

The factor  $Xw - y$  is the prediction-error vector. Multiplying by  $X^T$  aggregates how each parameter is responsible for those errors.

## LEARNING RATE INTUITION

- Too large: the update can overshoot the minimum, causing oscillation or divergence.
- Too small: the model moves in the correct direction but takes very small steps, so convergence becomes slow.
- Reasonable: loss generally decreases and the parameter updates become smaller as the optimum is approached.

## EVALUATION

$$\mathbf{MSE} = (1/N) \sum_i (\hat{y}_i - y_i)^2$$

$$\mathbf{RMSE} = \sqrt{\mathbf{MSE}}$$

$$\mathbf{MAE} = (1/N) \sum_i |\hat{y}_i - y_i|$$

$$\mathbf{R^2} = 1 - \mathbf{SS\_residual} / \mathbf{SS\_total}$$

The course emphasizes evaluating on unseen/test data because the real objective is generalization, not merely fitting the training examples.

## 6. REGRESSION NUMERICAL — FULL WALKTHROUGH

The uploaded Module 3 contains a small house-price-style example: size  $x_1 = 1, 2, 3$  and target  $y = 2, 4, 5$ . The learning rate is  $\eta = 0.1$  and the initial weight vector is zero.

### STEP 1 — WRITE THE MATRICES

$$\mathbf{X} = \begin{bmatrix} 1 & 1 \\ 1 & 2 \\ 1 & 3 \end{bmatrix}, \mathbf{y} = \begin{bmatrix} 2 \\ 4 \\ 5 \end{bmatrix}^T, \mathbf{w}^{(0)} = \begin{bmatrix} 0 \\ 0 \end{bmatrix}^T$$

The first column of  $\mathbf{X}$  contains ones for the bias term. This is why  $\mathbf{X}$  has two columns even though there is only one actual feature.

### STEP 2 — INITIAL PREDICTION

$$\hat{\mathbf{y}}^{(0)} = \mathbf{X}\mathbf{w}^{(0)} = \begin{bmatrix} 0 \\ 0 \\ 0 \end{bmatrix}^T$$

### STEP 3 — INITIAL LOSS

$$\begin{aligned} J(\mathbf{w}^{(0)}) &= (1/(2N)) \|\hat{\mathbf{y}}^{(0)} - \mathbf{y}\|^2 \\ &= (1/6)(2^2 + 4^2 + 5^2) = 7.5 \end{aligned}$$

The model initially predicts zero for every example, so the loss is large.

### STEP 4 — ERROR VECTOR

$$\mathbf{e}^{(0)} = \hat{\mathbf{y}}^{(0)} - \mathbf{y} = \begin{bmatrix} -2 \\ -4 \\ -5 \end{bmatrix}^T$$

### STEP 5 — GRADIENT

$$\nabla J(\mathbf{w}^{(0)}) = (1/3) \mathbf{X}^T \mathbf{e}^{(0)} = \begin{bmatrix} -3.67 \\ -8.33 \end{bmatrix}^T \text{ (approximately)}$$

Interpretation: both components are negative. Therefore the gradient-descent update subtracts negative values, increasing both parameters.

### STEP 6 — UPDATE

$$\begin{aligned} \mathbf{w}^{(1)} &= \mathbf{w}^{(0)} - 0.1 \nabla J(\mathbf{w}^{(0)}) \\ &\approx \begin{bmatrix} 0 \\ 0 \end{bmatrix}^T - 0.1 \begin{bmatrix} -3.67 \\ -8.33 \end{bmatrix}^T = \begin{bmatrix} 0.367 \\ 0.833 \end{bmatrix}^T \end{aligned}$$

### STEP 7 — NEW PREDICTIONS

$$\hat{\mathbf{y}}^{(1)} = \mathbf{X}\mathbf{w}^{(1)} \approx \begin{bmatrix} 1.20 \\ 2.03 \\ 2.87 \end{bmatrix}^T$$

### STEP 8 — NEW LOSS

$$J(\mathbf{w}^{(1)}) \approx 1.51$$

The uploaded material notes the drop from 7.5 to about 1.51 after one iteration. The numerical lesson is that one correct gradient step can dramatically reduce the error, but training continues until a stopping criterion is reached.

EXAM TIP: For any gradient-descent numerical, explicitly show: prediction  $\rightarrow$  error  $\rightarrow$  gradient  $\rightarrow$  update  $\rightarrow$  new prediction/loss. That sequence makes the logic visible and prevents sign errors.

COMMON TRAP: The course uses a  $\frac{1}{2}$  factor in its squared-loss formulation. If you use a different MSE convention, the numerical gradient changes by a constant factor. In an exam, follow the loss definition given in the question/lecture.

## 7. MODULE 4 — CLASSIFICATION

Classification predicts a discrete category rather than an unrestricted real number. The module distinguishes binary classification, multiclass classification and multilabel classification.

| Task                      | Output                             | Typical output mechanism in course |
|---------------------------|------------------------------------|------------------------------------|
| Regression                | Continuous real value              | Identity activation                |
| Binary classification     | Two classes / probability of class | Sigmoid                            |
| Multiclass classification | One of K classes                   | Softmax                            |

### WHY NOT USE LINEAR REGRESSION FOR CLASSIFICATION?

★★★★★

A linear regression model can produce any real number, including values below 0 and above 1. A probability should lie between 0 and 1. The solution is to take the linear score and pass it through a function that maps the real line into a probability range.

$$\mathbf{z} = \mathbf{w}^T \mathbf{x} + \mathbf{b}$$

$$\hat{y} = \sigma(\mathbf{z})$$

### LOGISTIC REGRESSION ★★★★★

Logistic regression is therefore a linear model underneath, followed by a nonlinear sigmoid mapping at the output. The boundary is still determined by  $\mathbf{z}=0$ , but the output is now a probability-like value.

$$\hat{y} = P(y=1|\mathbf{x}) = 1/(1+e^{(-\mathbf{z})})$$

### DECISION RULE ★★★★★

**Predict class 1 if  $\hat{y} \geq 0.5$ ; predict class 0 if  $\hat{y} < 0.5$**

Because  $\sigma(0)=0.5$ , this is equivalent to checking the sign of the linear score:

$$\mathbf{z} = \mathbf{w}^T \mathbf{x} + \mathbf{b} \geq 0 \Rightarrow \text{class 1}; \mathbf{z} < 0 \Rightarrow \text{class 0}$$

### BINARY CROSS-ENTROPY ★★★★★

$$\ell = -[y \log(\hat{y}) + (1-y) \log(1-\hat{y})]$$

The loss is designed around probability quality. If  $y=1$ , the first term matters and the model is rewarded for making  $\hat{y}$  close to 1. If  $y=0$ , the second term matters and the model is rewarded for making  $\hat{y}$  close to 0.

This explains why cross-entropy heavily penalizes confident mistakes. Predicting 0.99 for the wrong class is much worse than making a mildly uncertain prediction.

A particularly useful simplification occurs for sigmoid + BCE. The derivative with respect to the logit becomes:

$$\partial \ell / \partial \mathbf{z} = \hat{y} - y$$

$$\nabla_{\mathbf{w}} \ell = (\hat{y} - y) \mathbf{x}$$

So the gradient has an intuitive structure: prediction error  $\times$  input. This is one of the cleanest formulas to recognize in an exam.

## 8. MULTICLASS CLASSIFICATION — SOFTMAX + CROSS-ENTROPY

For  $K$  classes, the model produces  $K$  raw scores, often called logits. Softmax converts those scores into a probability distribution across the classes.

$$\hat{y}_k = e^{(z_k)} / \sum_j e^{(z_j)}$$

Two properties are crucial:

- Every probability is non-negative.
- All probabilities sum to 1.

$$\sum_k \hat{y}_k = 1$$

The outputs are therefore coupled: increasing one class's probability changes the normalization affecting the others. That is different from treating each class as an independent binary output.

### CATEGORICAL CROSS-ENTROPY ★★★★★

$$L = -\sum_k y_k \log(\hat{y}_k)$$

The course uses one-hot targets. If the correct class is class 2, the target vector is  $[0, 1, 0, \dots]$ . All terms except the correct-class term disappear because their target values are zero.

$$\text{If } \mathbf{y} = [0, 1, 0, 0], \text{ then } L = -\log(\hat{y}_2)$$

### SOFTMAX + CROSS-ENTROPY GRADIENT ★★★★★

$$\partial L / \partial \mathbf{z}_k = \hat{y}_k - y_k$$

This compact result is extremely exam-friendly. Once the softmax probabilities and one-hot target are known, subtract the target vector from the prediction vector.

Example from the uploaded numerical material:

$$\hat{\mathbf{y}} = [0.638, 0.235, 0.096, 0.032]$$

$$\mathbf{y} = [0, 1, 0, 0]$$

$$\partial L / \partial \mathbf{z} = \hat{\mathbf{y}} - \mathbf{y} = [0.638, -0.765, 0.096, 0.032]$$

The negative component for the correct class means the current probability for that class is too low relative to the target. Gradient descent will use this signal when changing the logits/weights.

### INFERENCE ★★★

$$\mathbf{z} = \mathbf{W}^T \mathbf{x}; \hat{\mathbf{y}} = \text{softmax}(\mathbf{z}); \text{predicted class} = \text{argmax}_k \hat{y}_k$$

During inference, there is no need to calculate a training loss unless you are evaluating the model. The basic prediction path is simply logits  $\rightarrow$  softmax  $\rightarrow$  largest probability.

## 9. CLASSIFICATION EVALUATION & DEBUGGING

### ACCURACY ★★★

$$\text{Accuracy} = \text{number of correct predictions} / N$$

Accuracy is intuitive but can be misleading when classes are highly imbalanced. The course also introduces confusion matrices and per-class precision/recall.

### CONFUSION MATRIX ★★★★★

For a K-class problem, the confusion matrix is  $K \times K$ . Rows represent actual classes and columns represent predicted classes in the course's convention. The diagonal contains correct predictions; off-diagonal entries are misclassifications.

| Actual \ Predicted | Class 1 | Class 2 | Class 3 |
|--------------------|---------|---------|---------|
| Class 1            | 45      | 3       | 2       |
| Class 2            | 5       | 38      | 7       |
| Class 3            | 2       | 4       | 44      |

Total correct =  $45 + 38 + 44 = 127$ . Total examples = 150. Therefore accuracy =  $127/150 \approx 84.7\%$ . The off-diagonal cells tell you which classes are being confused, which accuracy alone cannot show.

### PRECISION, RECALL & F1 ★★★

For a chosen class, think one-vs-all: the class of interest is 'positive' and all other classes are 'negative'.

$$\text{Precision} = TP / (TP + FP)$$

$$\text{Recall} = TP / (TP + FN)$$

$$F1 = 2(\text{Precision} \cdot \text{Recall}) / (\text{Precision} + \text{Recall})$$

The key conceptual distinction: precision asks how trustworthy the positive predictions are, while recall asks how many of the actual positives were found.

### DEBUGGING PATTERNS ★★★

| Observed problem                       | Likely causes / checks from course material                                |
|----------------------------------------|----------------------------------------------------------------------------|
| Loss is NaN/Inf                        | Learning rate too large; $\log(0)$ ; unstable softmax; feature scaling.    |
| Same class predicted everywhere        | Check class balance, softmax/loss, learning rate, one-hot encoding.        |
| Loss oscillates                        | Learning rate too large; consider mini-batches or learning-rate decay.     |
| Loss not decreasing                    | Check gradient signs, learning rate, feature scaling, shuffling, encoding. |
| High train accuracy, low test accuracy | Overfitting; use regularization, more data, or a simpler model.            |

**EXAM TIP:** If a numerical or implementation question asks you to diagnose training, start with the symptom. For example, oscillation strongly suggests an overly large learning rate in the uploaded material.

## 10. MODULE 5 — DEEP FEED-FORWARD NEURAL NETWORKS

A DFNN is presented in the course as a network with one or more hidden layers between input and output layers, where information flows forward through nonlinear transformations.

The important transition from Modules 3–4 is that the same framework is still present — data, model, loss and learning — but the model is now a stack of parameterized nonlinear layers.

A useful mental model is:

**Input → linear transformation → activation → linear transformation → activation → ... → output**

### LAYER-WISE FORWARD PROPAGATION ★★★★★

$$\mathbf{h}^{(0)} = \mathbf{x}$$

$$\mathbf{z}^{(l)} = \mathbf{h}^{(l-1)}\mathbf{W}^{(l)} + \mathbf{b}^{(l)}$$

$$\mathbf{h}^{(l)} = \sigma^{(l)}(\mathbf{z}^{(l)})$$

$$\hat{\mathbf{y}} = \mathbf{h}^{(L)}$$

Your uploaded Module 5 uses row-vector notation for its matrix dimensions, while the numerical practice material often displays column vectors. The underlying operation is the same: multiply the incoming activations by the layer's weight matrix, add the bias, and apply the layer activation. In an exam, follow the orientation used in the question and check dimensions carefully.

### WHY STORE $\mathbf{z}$ AND $\mathbf{h}$ ?

The forward pass produces intermediate values that are needed later by backpropagation. The pre-activation  $\mathbf{z}$  tells us which region of an activation function we are in, while  $\mathbf{h}$  is the output passed to the next layer. For ReLU, for example, the sign of  $\mathbf{z}$  determines whether the derivative is 0 or 1.

### MINI-BATCHES ★★★★★

Instead of processing only one example or the entire dataset at once, the course uses mini-batches. A mini-batch gives an average gradient that is more computationally efficient and can be parallelized, especially on GPUs.

$$\mathbf{J}_B = (1/B) \sum_{i \in B} \ell^{(i)}$$

$$\nabla \mathbf{J}_B = (1/B) \sum_{i \in B} \nabla \ell^{(i)}$$

The uploaded material lists common batch sizes such as 32, 64, 128 and 256 as examples. The conceptual trade-off is between computational efficiency and the stability of the gradient estimate.

### PARAMETER COUNT ★★★★★

$$\text{For layer } n_{(l-1)} \rightarrow n_l: \text{parameters} = n_{(l-1)}n_l + n_l = n_l(n_{(l-1)}+1)$$

$$\text{Total parameters} = \sum_l n_l(n_{(l-1)}+1)$$

Why add  $n_l$ ? Because each of the  $n_l$  neurons has one bias. The first term counts all incoming weights; the second counts one bias per output neuron.

Example from the course: architecture  $784 \rightarrow 256 \rightarrow 128 \rightarrow 10$  has:

**Layer 1:  $784 \times 256 + 256 = 200,960$**

**Layer 2:  $256 \times 128 + 128 = 32,896$**

**Layer 3:  $128 \times 10 + 10 = 1,290$**

**Total = 235,146 parameters**

## 11. DFNN NUMERICAL — FORWARD PROPAGATION

The uploaded DFNN numerical practice contains a complete two-layer example. The network has two inputs, three hidden ReLU units and two sigmoid outputs.

### Given:

$$\mathbf{x} = [1.0, 0.5]^T$$

$$\mathbf{y} = [1, 0]^T$$

$$\mathbf{W}^1 = [[0.5, 0.2], [-0.3, 0.8], [0.1, -0.4]], \mathbf{b}^1 = [0.1, -0.2, 0.3]^T$$

$$\mathbf{W}^2 = [[0.4, -0.1, 0.6], [0.2, 0.7, -0.3]], \mathbf{b}^2 = [0.2, -0.1]^T$$

### STEP 1 — FIRST LAYER PRE-ACTIVATION

$$\mathbf{z}^1 = \mathbf{W}^1 \mathbf{x} + \mathbf{b}^1$$

$$= [0.5(1) + 0.2(0.5) + 0.1, -0.3(1) + 0.8(0.5) - 0.2, 0.1(1) - 0.4(0.5) + 0.3]^T$$

$$= [0.7, -0.1, 0.2]^T$$

### STEP 2 — APPLY ReLU

$$\mathbf{a}^1 = \text{ReLU}(\mathbf{z}^1) = [0.7, 0, 0.2]^T$$

The second hidden neuron becomes zero because its pre-activation is negative. This is exactly the type of intermediate calculation shown in the uploaded numerical material.

### STEP 3 — SECOND LAYER PRE-ACTIVATION

$$\mathbf{z}^2 = \mathbf{W}^2 \mathbf{a}^1 + \mathbf{b}^2$$

$$\text{First output} = 0.4(0.7) + (-0.1)(0) + 0.6(0.2) + 0.2 = 0.6$$

$$\text{Second output} = 0.2(0.7) + 0.7(0) + (-0.3)(0.2) - 0.1 = -0.02$$

$$\mathbf{z}^2 = [0.6, -0.02]^T$$

### STEP 4 — APPLY SIGMOID

$$\hat{\mathbf{y}} = \sigma(\mathbf{z}^2) \approx [0.646, 0.495]^T$$

So the forward pass is complete. The important thing is the chain:

$$\mathbf{x} \rightarrow \mathbf{z}^1 \rightarrow \text{ReLU} \rightarrow \mathbf{a}^1 \rightarrow \mathbf{z}^2 \rightarrow \text{sigmoid} \rightarrow \hat{\mathbf{y}}$$

EXAM TIP: In a forward-propagation numerical, never try to compute the whole network in one line. Write each  $\mathbf{z}$  and each activation separately. The intermediate values are also what you need for backpropagation.

## 12. BACKPROPAGATION — THE CONCEPTUAL HEART

Backpropagation is not a different optimization method from gradient descent. It is the efficient method used to calculate gradients of the loss with respect to all the weights and biases in a multilayer network. Gradient descent then uses those gradients to update the parameters.

The course describes backpropagation as applying the chain rule backward through the network. That wording is the key to understanding it.

### WHY THE CHAIN RULE?

Suppose the loss depends on an output, the output depends on a hidden activation, and that activation depends on a weight. Then a change in the weight affects the loss through several intermediate quantities. The chain rule lets us multiply the local derivatives along that path.

**If  $L$  depends on  $y$ ,  $y$  depends on  $z$ , and  $z$  depends on  $w$ :  $\partial L / \partial w = (\partial L / \partial y)(\partial y / \partial z)(\partial z / \partial w)$**

Instead of separately recomputing every derivative from scratch, backpropagation reuses error signals already computed at later layers. That is why it scales to deep networks.

### ERROR SIGNAL / DELTA ★★★★★

$$\delta^{(l)} = \partial J / \partial z^{(l)}$$

Think of  $\delta$  as the sensitivity of the loss to that layer's pre-activation. Once  $\delta$  is known, both the weight gradient and bias gradient can be built from it.

### OUTPUT-LAYER GRADIENT ★★★★★

For the three paired loss/activation combinations emphasized in the course — MSE + identity, BCE + sigmoid, and CCE + softmax — the output-layer gradient takes the elegant form:

$$\delta^{(L)} = (1/B)(\hat{Y} - Y) \text{ for a mini-batch}$$

For a single example, the  $1/B$  factor disappears. This is why the same-looking 'prediction minus target' pattern appears repeatedly across regression and classification.

### HIDDEN-LAYER BACKPROPAGATION ★★★★★

$$\delta^{(l)} = [\delta^{(l+1)}(W^{(l+1)})^T] \odot \sigma'(z^{(l)})$$

Read this from left to right conceptually: take the error from the next layer → transport it backward through the next layer's weights → multiply element-wise by the current activation derivative.

The Hadamard symbol  $\odot$  means element-wise multiplication. This matters because the activation derivative is applied separately to each neuron.

### PARAMETER GRADIENTS ★★★★★

$$\partial J / \partial W^{(l)} = (1/B)(H^{(l-1)})^T \delta^{(l)}$$

$$\partial J / \partial b^{(l)} = (1/B) \mathbf{1}_B^T \delta^{(l)}$$

For one example, the weight-gradient idea is simply an outer product: incoming activation  $\times$  current error signal. The bias gradient is the error signal itself.

Finally:

$$\mathbf{W}^{(l)} \leftarrow \mathbf{W}^{(l)} - \eta \frac{\partial J}{\partial \mathbf{W}^{(l)}}$$

$$\mathbf{b}^{(l)} \leftarrow \mathbf{b}^{(l)} - \eta \frac{\partial J}{\partial \mathbf{b}^{(l)}}$$

## 13. DFNN NUMERICAL — BACKPROPAGATION WALKTHROUGH

Continue the exact uploaded numerical example. The forward pass gave:

$$\mathbf{a}^2 = \hat{\mathbf{y}} = [0.646, 0.495]^T, \mathbf{y} = [1, 0]^T$$

### STEP 1 — BCE LOSS

$$\begin{aligned} L &= -\frac{1}{2}[1 \cdot \log(0.646) + 0 \cdot \log(0.495) + 0 \cdot \log(1 - 0.646) + 1 \cdot \log(1 - 0.495)] \\ &\approx -\frac{1}{2}[\log(0.646) + \log(0.505)] \approx 0.560 \end{aligned}$$

The uploaded numerical material reports approximately 0.560.

### STEP 2 — OUTPUT ERROR SIGNAL

$$\delta^2 = \mathbf{a}^2 - \mathbf{y} = [0.646 - 1, 0.495 - 0]^T = [-0.354, 0.495]^T$$

The negative first component says the first output needs to move upward toward its target of 1; the positive second component says the second output is too high relative to its target of 0.

### STEP 3 — OUTPUT-LAYER WEIGHT GRADIENT

$$\begin{aligned} \partial L / \partial \mathbf{W}^2 &= \delta^2 (\mathbf{a}^1)^T \\ &= [-0.354, 0.495]^T [0.7, 0, 0.2] \\ &= [[-0.248, 0, -0.071], [0.347, 0, 0.099]] \text{ (approximately)} \end{aligned}$$

Notice the entire middle column is zero. That is because the second hidden activation was zero. This is a beautiful numerical illustration of how a ReLU unit with zero activation contributes no immediate outer-product term to the next layer's weight gradient.

$$\partial L / \partial \mathbf{b}^2 = \delta^2 = [-0.354, 0.495]^T$$

### STEP 4 — PROPAGATE ERROR TO HIDDEN LAYER

$$(\mathbf{W}^2)^T \delta^2 = [-0.043, 0.382, -0.361]^T$$

Now apply the ReLU derivative. Since  $\mathbf{z}^1 = [0.7, -0.1, 0.2]$ , its derivative is  $[1, 0, 1]$ .

$$\text{ReLU}'(\mathbf{z}^1) = [1, 0, 1]^T$$

$$\delta^1 = [-0.043, 0.382, -0.361] \odot [1, 0, 1] = [-0.043, 0, -0.361]^T$$

The middle hidden neuron's error becomes zero because its ReLU derivative is zero. This is exactly why remembering both  $\mathbf{z}$  and  $\mathbf{a}$  from the forward pass matters.

### STEP 5 — FIRST-LAYER GRADIENTS

$$\begin{aligned} \partial L / \partial \mathbf{W}^1 &= \delta^1 \mathbf{x}^T \\ &= [-0.043, 0, -0.361]^T [1, 0.5] \\ &\approx [[-0.043, -0.022], [0, 0], [-0.361, -0.181]] \\ \partial L / \partial \mathbf{b}^1 &= [-0.043, 0, -0.361]^T \end{aligned}$$

## STEP 6 — PARAMETER UPDATE

With  $\eta=0.1$ , the uploaded numerical gives the updated second-layer parameters:

$$\mathbf{W}^2(\text{new}) \approx [[0.425, -0.100, 0.607], [0.165, 0.700, -0.310]]$$

$$\mathbf{b}^2(\text{new}) \approx [0.235, -0.150]^T$$

The update is always old minus learning-rate times gradient. The signs matter. If the gradient is negative, subtracting it increases the corresponding parameter.

EXAM TIP: Backprop numerical template: loss  $\rightarrow \delta$  at output  $\rightarrow W/b$  gradients at output  $\rightarrow$  backpropagate  $\delta \rightarrow$  multiply by activation derivative  $\rightarrow$  hidden gradients  $\rightarrow$  update.

## 14. DEPTH, WIDTH & ARCHITECTURE DESIGN

### DEPTH ★★★★★

Depth is the number of layers contributing to the model, with the course's DFNN notation counting layers from the first hidden/output transformation onward. Depth allows representations to be built hierarchically.

### WIDTH ★★★★★

Width is the number of units in a layer. More width increases the capacity of that layer to represent patterns, but it also increases parameter count and can increase data requirements.

| Increasing depth                      | Increasing width                           |
|---------------------------------------|--------------------------------------------|
| More successive transformations       | More units per transformation              |
| Supports hierarchical representations | Increases capacity within a layer          |
| Can capture multi-stage structure     | Can provide more representational capacity |

### UNIVERSAL APPROXIMATION — WHAT TO SAY IN AN EXAM

★★★

The course material states that a feed-forward network with a single hidden layer and suitable activation can approximate a broad class of continuous functions under appropriate conditions. The practical point emphasized is that a shallow network may need a very large hidden layer, while deeper networks can represent hierarchical structure more naturally.

### ARCHITECTURE DESIGN ★★★

When designing a DFNN, think in terms of constraints: input dimension, output dimension, amount of data, parameter count, computational resources, and the complexity of the task. The uploaded numerical practice shows examples where an apparently reasonable deep architecture has far too many parameters for the dataset, so a smaller architecture is selected.

The course's numerical material gives a text-data example where a naive architecture would have about 17 million parameters and is rejected under a parameter/data constraint. A more modest [1000,512,256,128,5] architecture is then calculated at roughly 675,000 parameters and considered more reasonable in that example.

The exam lesson: architecture is a design decision, not just a mathematical calculation. You should be able to explain why more neurons/layers increase capacity but also increase computational cost and the risk of overfitting.

## 15. INITIALIZATION, SCALING & TRAINING PRACTICE

### WEIGHT INITIALIZATION ★★★★★

The uploaded DFNN material emphasizes that initialization matters. It recommends random initialization for weights rather than making all neurons identical, and it mentions Xavier/Glorot initialization for sigmoid/tanh and He initialization for ReLU.

The intuition is symmetry breaking. If every neuron starts with exactly the same weights and receives the same inputs, there is little reason for those neurons to learn different representations. Randomized initial weights allow different neurons to develop different roles.

The material allows zero initialization for biases. The regression module also notes that zero initialization can work well for linear regression; the more important warning in deep networks is to avoid symmetric zero initialization of all weights.

### FEATURE SCALING ★★★★★

Features with very different numerical scales can make optimization poorly conditioned. The regression module gives two scaling examples:

$$\text{Min-Max: } x'_j = [x_j - \min(x_j)] / [\max(x_j) - \min(x_j)]$$

$$\text{Z-score: } x'_j = (x_j - \mu_j) / \sigma_j$$

The idea is not to change the underlying information; it is to put features on a more manageable numerical scale so that gradient-based optimization can make progress more efficiently.

### STOPPING TRAINING ★★★★★

- Gradient norm becomes small:  $\|\nabla\| < \epsilon$ .
- Loss change between iterations becomes very small.
- A maximum number of iterations/epochs is reached.
- Validation loss starts increasing, suggesting overfitting; early stopping can then be used.

### GENERALIZATION ★★★★★

A model that memorizes training data but performs poorly on unseen data has poor generalization. The course's regression material explicitly distinguishes training error from test error and emphasizes evaluating on unseen examples.

Common responses to overfitting mentioned across the material include regularization, more data, simpler models, dropout/batch normalization in architecture examples, and early stopping.

## 16. MASTER FORMULA SHEET — MEMORIZE + UNDERSTAND

| Topic              | Formula                                                               | What it means                            |
|--------------------|-----------------------------------------------------------------------|------------------------------------------|
| Artificial neuron  | $z = w^T x + b; \hat{y} = f(z)$                                       | Weighted combination + bias + activation |
| Perceptron         | $\hat{y} = 1$ if $w^T x + b \geq 0$ , else 0                          | Linear threshold classifier              |
| Perceptron update  | $w_i \leftarrow w_i + \eta(t - \hat{y})x_i$                           | Correct a misclassification              |
| Linear regression  | $\hat{y} = w^T x + b$                                                 | Continuous output                        |
| Identity           | $f(z) = z$                                                            | Leaves regression output unrestricted    |
| Squared loss       | $\ell = \frac{1}{2}(\hat{y} - y)^2$                                   | Penalizes large errors quadratically     |
| MSE gradient       | $\nabla J = (1/N)X^T(Xw - y)$                                         | Direction of increasing loss             |
| GD update          | $w \leftarrow w - \eta \nabla J$                                      | Move toward lower loss                   |
| Sigmoid            | $\sigma(z) = 1/(1 + e^{-z})$                                          | Maps real value to (0,1)                 |
| Sigmoid derivative | $\sigma' = \sigma(1 - \sigma)$                                        | Needed for backprop                      |
| BCE                | $-[y \log \hat{y} + (1 - y) \log(1 - \hat{y})]$                       | Binary classification loss               |
| Binary gradient    | $\nabla_w L = (\hat{y} - y)x$                                         | Error $\times$ input                     |
| Softmax            | $\hat{y}_k = e^{z_k} / \sum_j e^{z_j}$                                | Probability distribution over K classes  |
| CCE                | $-\sum_k y_k \log \hat{y}_k$                                          | Multiclass classification loss           |
| Softmax+CCE        | $\partial L / \partial z_k = \hat{y}_k - y_k$                         | Logit gradient                           |
| ReLU               | $\max(0, z)$                                                          | Common hidden-layer activation           |
| ReLU derivative    | 1 if $z > 0$ ; 0 if $z \leq 0$                                        | Gate for backprop                        |
| DFNN layer         | $z^{(l)} = h^{(l-1)}W^{(l)} + b^{(l)}$                                | Pre-activation                           |
| DFNN activation    | $h^{(l)} = \sigma^{(l)}(z^{(l)})$                                     | Layer output                             |
| Hidden delta       | $\delta^{(l)} = [\delta^{(l+1)}(W^{(l+1)})^T] \odot \sigma'(z^{(l)})$ | Backpropagate error                      |
| Weight gradient    | $\partial J / \partial W^{(l)} = (1/B)(H^{(l-1)})^T \delta^{(l)}$     | How weights affect loss                  |
| Bias gradient      | $\partial J / \partial b^{(l)} = (1/B)1^T \delta^{(l)}$               | How bias affects loss                    |
| Parameter update   | $\theta \leftarrow \theta - \eta \partial J / \partial \theta$        | Learning step                            |
| Parameter count    | $n_l(n_{l-1} + 1)$                                                    | Weights + biases in one layer            |
| Accuracy           | $\text{correct}/N$                                                    | Overall correctness                      |
| Precision          | $TP/(TP+FP)$                                                          | Trustworthiness of positive predictions  |
| Recall             | $TP/(TP+FN)$                                                          | Fraction of actual positives found       |
| F1                 | $2PR/(P+R)$                                                           | Balance of precision and recall          |

**COMMON TRAP:** Do not memorize formulas as isolated symbols. For every formula, know what problem it solves, what each symbol means, and what direction the update moves.

## 17. NUMERICAL TEMPLATES — WHAT TO WRITE IN THE EXAM

### TEMPLATE A — SINGLE NEURON

- 1 Write  $z = w^T x + b$ .
- 2 Substitute every input/weight explicitly.
- 3 Calculate  $z$ .
- 4 Apply the requested activation  $f(z)$ .
- 5 If classification, apply the decision rule.

**Example structure:  $z = (w_1 x_1 + w_2 x_2 + \dots) + b \rightarrow \hat{y} = f(z)$**

### TEMPLATE B — PERCEPTRON LEARNING

- 1 Initialize  $\eta$  and weights/bias.
- 2 For each training example calculate the weighted sum.
- 3 Calculate the threshold prediction.
- 4 Compare prediction with target.
- 5 If wrong, update every weight and the bias.
- 6 Repeat through epochs until convergence or the stated stopping condition.

### TEMPLATE C — BINARY CLASSIFICATION

- 1  $z = w^T x + b$
- 2  $\hat{y} = \text{sigmoid}(z)$
- 3 Class = 1 if  $\hat{y} \geq 0.5$ , otherwise 0.
- 4  $\text{BCE} = -[y \log \hat{y} + (1-y) \log (1-\hat{y})]$ .
- 5 Gradient =  $(\hat{y} - y)x$ .
- 6 Update  $w$  and  $b$  using gradient descent.

### TEMPLATE D — MULTICLASS

- 1 Calculate all logits  $z_1, \dots, z_K$ .
- 2 Calculate softmax probabilities.
- 3 Check that probabilities sum to approximately 1.
- 4 Prediction = argmax probability.
- 5  $\text{CCE} = -\sum y_k \log \hat{y}_k$ .
- 6 Gradient wrt logits =  $\hat{y} - y$ .

### TEMPLATE E — DFNN FORWARD PASS

- 1 Set  $h^{(0)} = x$ .
- 2 For each layer calculate  $z^{(l)}$ .

- 3 Apply the layer activation to obtain  $h^{(l)}$ .
- 4 Continue until  $h^{(L)} = \hat{y}$ .
- 5 Keep all intermediate  $z$  and  $h$  values.

## TEMPLATE F — DFNN BACKPROP

- 1 Calculate the loss.
- 2 Find output  $\delta$ .
- 3 Compute output-layer weight and bias gradients.
- 4 Backpropagate  $\delta$  to the previous layer.
- 5 Multiply by the activation derivative.
- 6 Compute hidden-layer gradients.
- 7 Update  $W$  and  $b$  with  $\theta \leftarrow \theta - \eta \partial J / \partial \theta$ .

## 18. LIKELY EXAM QUESTIONS — HIGH-YIELD SET

These are prioritized from the structure and emphasis of the uploaded modules. They are not a claim about the exact wording of the future paper.

### TIER 1 — MUST MASTER ★★★★★

- 1 Explain deep learning and why the term 'deep' is used.
- 2 Differentiate AI, ML and DL.
- 3 Explain the four components of a deep-learning problem.
- 4 Explain an artificial neuron, including weights and bias.
- 5 Explain the perceptron learning rule with a numerical example.
- 6 Why can a perceptron solve AND/OR but not XOR?
- 7 Explain linear separability and decision boundaries.
- 8 Why are nonlinear activation functions essential in DFNNs?
- 9 Compare sigmoid, tanh and ReLU, including derivatives and practical use.
- 10 Derive/use the gradient-descent update rule.
- 11 Explain linear regression as a single neuron with identity activation.
- 12 Explain why MSE is suitable for the course's linear regression formulation.
- 13 Explain logistic regression and the role of sigmoid.
- 14 Calculate BCE for a binary example.
- 15 Derive/use the binary classification gradient  $(\hat{y}-y)x$ .
- 16 Explain softmax and why its outputs form a probability distribution.
- 17 Calculate softmax for given logits.
- 18 Calculate categorical cross-entropy for a one-hot target.
- 19 Show that the softmax + cross-entropy logit gradient is  $\hat{y}-y$ .
- 20 Perform a complete DFNN forward propagation.
- 21 Explain backpropagation using the chain rule.
- 22 Calculate  $\delta$  for a hidden layer.
- 23 Calculate weight and bias gradients.
- 24 Perform one complete parameter update.
- 25 Count trainable parameters in a multilayer architecture.

### TIER 2 — VERY LIKELY / GOOD REVISION ★★★★★

- 1 Regression vs classification.
- 2 Identity vs sigmoid vs softmax output activations.
- 3 MSE vs MAE and sensitivity to outliers.
- 4 Training error vs test error and generalization.
- 5 Depth vs width.

- 6 Why random initialization is used for deep networks.
- 7 Xavier/Glorot vs He initialization — what the course associates them with.
- 8 Why feature scaling helps gradient-based optimization.
- 9 Explain mini-batch SGD.
- 10 Interpret a confusion matrix.
- 11 Precision vs recall vs F1.
- 12 Diagnose NaN loss, oscillating loss, or high train/low test accuracy.

## 19. THEORY ANSWER FRAMEWORKS — WRITE LIKE AN EXAM ANSWER

### Q: Why can't a single perceptron solve XOR?

Answer framework: A single perceptron produces a linear decision boundary of the form  $w^T x + b = 0$ . The XOR data are not linearly separable, so no single straight line/hyperplane can separate the two classes. Therefore a multilayer network with nonlinear activations is required. The hidden layer can create a new representation in which the problem becomes separable.

### Q: Why are nonlinear activation functions necessary?

Answer framework: If every layer is linear, the composition of multiple linear transformations is still a single linear transformation. Therefore simply adding linear layers does not increase nonlinear expressive power. Nonlinear activations prevent this collapse and allow the network to model complex nonlinear relationships.

### Q: Explain gradient descent.

Answer framework: Gradient descent is an iterative optimization method. The gradient points in the direction of increasing loss, so the parameters are updated in the opposite direction:  $\theta \leftarrow \theta - \eta \nabla J$ . The learning rate controls the step size. A very large rate may overshoot or oscillate; a very small rate may make convergence slow.

### Q: Explain backpropagation.

Answer framework: Backpropagation efficiently computes the gradient of the loss with respect to the parameters of a multilayer network by applying the chain rule backward from the output layer. It first computes the output error signal, recursively propagates error signals to hidden layers, then uses those signals to calculate weight and bias gradients. Gradient descent uses those gradients to update the parameters.

### Q: Why use softmax for multiclass classification?

Answer framework: Softmax converts K logits into non-negative values that sum to one, allowing them to be interpreted as a probability distribution across mutually exclusive classes. The predicted class is typically the class with the largest probability, and the course pairs softmax with categorical cross-entropy.

### Q: Why is ReLU popular in hidden layers?

Answer framework: ReLU is computationally simple, gives a derivative of 1 for positive inputs, supports efficient gradient flow on that side, and produces sparse activations. The course also notes the dying-ReLU caveat and mentions variants such as Leaky ReLU, PReLU and ELU.

## 20. EXAM QUIZ — 30 QUESTIONS

Try these without looking at the answer key. For numerical questions, write the intermediate steps, not just the final number.

**1. What does 'deep' refer to in deep learning?**

Answer: The number of successive layers/transformations contributing to learned representations.

**2. What are the four core components of a learning problem?**

Answer: Data, model, objective/loss function, and learning algorithm.

**3. Write the artificial-neuron equation.**

Answer:  $z = w^T x + b$  and  $\hat{y} = f(z)$ .

**4. What does a weight represent conceptually?**

Answer: The learned influence/strength associated with an input connection.

**5. What does the bias do?**

Answer: It provides an adjustable offset to the weighted sum and therefore shifts the decision/function.

**6. State the perceptron learning rule.**

Answer:  $w_i(\text{new}) = w_i(\text{old}) + \eta(t - \hat{y})x_i$ ;  $b(\text{new}) = b(\text{old}) + \eta(t - \hat{y})$ .

**7. Why does a single perceptron fail on XOR?**

Answer: XOR is not linearly separable; one perceptron creates only a linear boundary.

**8. Why do we need nonlinear activation functions?**

Answer: Without them, stacked linear layers collapse to one linear transformation.

**9. Write the sigmoid function.**

Answer:  $\sigma(z) = 1/(1 + e^{(-z)})$ .

**10. If  $\sigma(z) = 0.8$ , what is  $\sigma'(z)$ ?**

Answer:  $0.8(1 - 0.8) = 0.16$ .

**11. Write ReLU and its derivative.**

Answer:  $\text{ReLU}(z) = \max(0, z)$ ; derivative is 1 for  $z > 0$  and 0 for  $z \leq 0$ .

**12. Which activation is used for the course's linear regression output?**

Answer: Identity:  $f(z) = z$ .

**13. Write the gradient-descent update.**

Answer:  $\theta \leftarrow \theta - \eta \nabla J$ .

**14. What happens if  $\eta$  is too large?**

Answer: The updates may overshoot, oscillate, or diverge.

**15. What is the binary cross-entropy loss?**

Answer:  $-[y \log \hat{y} + (1 - y) \log(1 - \hat{y})]$ .

**16. What is the binary logistic-regression gradient wrt weights?**

Answer:  $(\hat{y} - y)x$ .

**17. Why is 0.5 the usual sigmoid decision threshold?**

Answer: Because  $\text{sigmoid}(0) = 0.5$ , so it is equivalent to checking whether the linear score is nonnegative.

**18. Write the softmax formula.**

Answer:  $\hat{y}_k = e^{z_k} / \sum_j e^{z_j}$ .

**19. What property must softmax outputs satisfy?**

Answer: They are nonnegative and sum to 1.

**20. Write categorical cross-entropy.**

Answer:  $-\sum_k y_k \log \hat{y}_k$ .

**21. What is the softmax + CCE gradient wrt logits?**

Answer:  $\hat{y} - y$ .

**22. If  $\hat{y}=[0.2,0.7,0.1]$ , what class is predicted?**

Answer: Class 2.

**23. If  $y=[0,1,0,0]$  and  $\hat{y}_2=0.25$ , what is the CCE for that one-hot example?**

Answer:  $-\log(0.25) \approx 1.386$ .

**24. What does  $\delta^{(l)}$  represent?**

Answer: The derivative of the loss with respect to the layer's pre-activation  $z^{(l)}$ .

**25. Write the hidden-layer backprop equation.**

Answer:  $\delta^{(l)} = [\delta^{(l+1)} (W^{(l+1)})^T] \odot \sigma'(z^{(l)})$ .

**26. What is the single-example weight-gradient structure?**

Answer: Incoming activation transposed times the current error signal:  $(a^{(l-1)})^T \delta^{(l)}$ , with orientation adjusted to the notation.

**27. For 10 inputs and 5 output neurons, how many parameters?**

Answer:  $10 \times 5$  weights + 5 biases = 55.

**28. What does high train accuracy but low test accuracy suggest?**

Answer: Overfitting.

**29. What is the difference between precision and recall?**

Answer: Precision asks how many predicted positives are correct; recall asks how many actual positives were found.

**30. What are the major steps of a DFNN training cycle?**

Answer: Forward pass  $\rightarrow$  loss  $\rightarrow$  backpropagation  $\rightarrow$  gradients  $\rightarrow$  parameter update  $\rightarrow$  repeat.

## 21. PRACTICE NUMERICALS — DO THESE ON PAPER

### A. Single neuron

$x=[2,-1]^T$ ,  $w=[0.4,0.7]^T$ ,  $b=-0.2$ . Calculate  $z$  and the sigmoid output.

### B. ReLU

$z=[2.0,-1.5,0.8,-0.3]^T$ . Calculate  $\text{ReLU}(z)$  and  $\text{ReLU}'(z)$ .

### C. Perceptron update

$\eta=0.1$ ,  $x=[2,1]^T$ , target  $t=1$ , prediction  $\hat{y}=0$ . Starting  $w=[-0.2,0.4]^T$  and  $b=0.1$ , calculate the updated parameters.

### D. Regression loss

Targets  $y=[2,4,5]$  and predictions  $\hat{y}=[1.2,2.03,2.87]$ . Calculate the squared-error/MSE-style loss using the convention stated in the question.

### E. Binary cross-entropy

$y=0$  and  $\hat{y}=0.2$ . Calculate BCE.

### F. Softmax

Logits  $z=[1.0,2.0,0.0]$ . Calculate the three softmax probabilities and identify the predicted class.

### G. Multiclass gradient

$\hat{y}=[0.2,0.5,0.3]$ ,  $y=[0,1,0]$ . Calculate  $\partial L/\partial z$ .

### H. Parameter update

$w=1.5$ ,  $\eta=0.05$ ,  $\text{gradient}=-2.0$ . Calculate  $w(\text{new})$ .

### I. Parameter counting

Architecture  $20 \rightarrow 10 \rightarrow 5 \rightarrow 2$ . Calculate the total number of weights, biases and trainable parameters.

### J. DFNN forward pass

Use a two-layer network and calculate every  $z$  and activation. Show dimensions at every matrix multiplication.

### K. DFNN backprop

Given output probabilities and a one-hot target, calculate  $\delta$  at the output, then backpropagate through a ReLU hidden layer and calculate the first-layer gradients.

## SELF-CHECK ROUTINE

- Did every matrix multiplication have compatible dimensions?
- Did you add the bias to the correct neuron?
- Did you apply the correct activation at the correct layer?

- For ReLU, did you use  $z$  — not the post-activation value — to determine the derivative?
- For softmax, do the probabilities sum to approximately 1?
- For one-hot cross-entropy, did only the correct class contribute?
- Did you subtract the gradient in the parameter update?
- Did you average by batch size when the question specifies a mini-batch?

## 22. FINAL 30-MINUTE CRAM PAGE

If you have only 30 minutes before the exam, do this in order:

- 1 5 min: Recite the master formulas from memory.
- 2 5 min: Explain XOR and why nonlinearity is needed.
- 3 5 min: Do one sigmoid/BCE numerical and one softmax/CCE numerical.
- 4 10 min: Do a complete DFNN forward + backprop numerical.
- 5 5 min: Review likely questions and common traps.

### THE 12 FORMULAS I WOULD NOT ENTER THE EXAM WITHOUT KNOWING

1.  $\mathbf{z} = \mathbf{w}^T \mathbf{x} + \mathbf{b}$
2.  $\hat{y} = f(\mathbf{z})$
3.  $\sigma(\mathbf{z}) = 1 / (1 + e^{(-\mathbf{z})})$
4.  $\sigma'(\mathbf{z}) = \sigma(\mathbf{z})(1 - \sigma(\mathbf{z}))$
5.  $\text{ReLU}(\mathbf{z}) = \max(0, \mathbf{z})$
6. MSE-style loss:  $(1/(2N)) ||\mathbf{Xw} - \mathbf{y}||^2$
7.  $\nabla J = (1/N) \mathbf{X}^T (\mathbf{Xw} - \mathbf{y})$
8.  $\theta \leftarrow \theta - \eta \nabla J$
9. BCE =  $-[y \log \hat{y} + (1-y) \log (1-\hat{y})]$
10. softmax:  $\hat{y}_k = e^{(\mathbf{z}_k)} / \sum_j e^{(\mathbf{z}_j)}$
11. CCE =  $-\sum_k y_k \log \hat{y}_k$ ;  $\partial L / \partial \mathbf{z} = \hat{\mathbf{y}} - \mathbf{y}$
12.  $\delta_{\text{hidden}} = [\delta_{\text{next}} \mathbf{W}_{\text{next}}^T] \odot \sigma'(\mathbf{z}_{\text{hidden}})$

### THE ONE STORY TO REMEMBER

**A neuron computes a weighted sum → activation creates nonlinearity → layers create representations → loss measures error → backprop computes responsibility/gradients → gradient descent changes parameters.**

If you understand that sentence, the formulas stop looking like unrelated equations.

EXAM TIP: In the exam, show your work. A correct setup with clear intermediate values can earn substantial partial credit even if an arithmetic slip occurs later.

## 23. SOURCE MAP — WHERE THE MAIN IDEAS COME FROM

| Course material                         | Main ideas used in this pack                                                                                    |
|-----------------------------------------|-----------------------------------------------------------------------------------------------------------------|
| DNN_M1_Introduction                     | Deep learning, AI/ML/DL, why DL, applications, data/model/loss/learning algorithm, supervised learning          |
| DNN_M2_ANN_Perceptron                   | Biological/artificial neuron, perceptron, AND/OR, perceptron learning, XOR, linear separability.                |
| DNN_M3_Linear NN Regression             | Linear regression, identity activation, squared error, matrix formulation, gradient descent, worked numerical   |
| DNN_M4_Linear NN Classification         | Binary classification, logistic regression, sigmoid, BCE, SGD, multiclass softmax, CCE, confusion matrix        |
| DNN_M5_DFNN                             | XOR motivation, nonlinearity, DFNN architecture, mini-batches, parameter counting, forward propagation          |
| DFNN NUMERICAL / DNN_M5_DFNN_Numericals | Step-by-step activation, forward propagation, BCE, output error, hidden error, gradients and parameter counting |

Important note: the mathematical explanations in this pack are intentionally expanded to make the course ideas easier to understand, but the terminology and core equations follow the uploaded material.

## END OF PACK

Good luck — and remember: for DNN numericals, write the computation as a sequence. Do not try to jump directly from the input to the final answer.